Estrutura de Dados II

Prof. Vanessa Matias Leite

1

Compressão de Dados

2

Compressão de Dados

- Consiste na utilização de um conjunto de métodos com o intuito da redução do espaço armazenado em unidades de memória secundária ou mesmo primária de um sistema computacional.
- Um arquivo comprimido terá seu tamanho reduzido, como saída resultante da aplicação de um algoritmo de compactação de alguma aplicação, como, por exemplo, gzip, winzip ou winrar.

3

Compressão de Dados

- Na internet, a compressão ajuda a diminuir a quantidade de tráfego na grande rede, aumentando a velocidade de navegação, realização de downloads de arquivos e visualização de vídeos.
- Arquivos compactados são preferíveis quando há interesse de armazenamento de maior número de dados possível no menor espaço de memória secundária disponível, como em pen-drives, memmory cards, discos rígidos e demais unidades de memória.

4

Compressão de Dados

- Muitos arquivos de extensões famosas, como pdf (textos, e-books), mp3 (áudio, músicas), gif (imagens, fotos), zip (arquivos em geral), mpg (vídeos), utilizam algoritmos de compactação em suas concepções.
- Arquivos são **sequências de bytes**, caracteres alfabéticos, numéricos e símbolos imprimíveis ou não. Se começarmos a raciocinar sobre essas sequências, é provável que comecemos a imaginar **maneiras de reorganizar ou representar tais sequências de uma maneira que a quantidade de bytes possa ser reduzida**.

5

Compressão x Compactação

Compressão
- Mudança na representação de algum dado para reduzir seu tamanho

Compactação:
- União de dados que não estão unidos

Exemplo:
- Compressão do HD: reduzir o tamanho dos arquivos
- Compactação do HD: juntar partes disjuntas (desfragmentar)

6

Definição de Compressão

- Representação de uma fonte de dados da maneira mais precisa possível utilizando um menor número de bits;
- Fazer com que a mesma quantidade de informação caiba em um espaço menor;
- Eliminar as redundâncias: recorrências de letras, dígitos ou pixels;
- Receptor deve ser capaz de decodificar os dados para acessar a informação;

Exemplos:

- 'AAAAAA', que ocupa 6 bytes, poderia ser comprimida para '6A', que ocupa 2 bytes);
- AATTTT representa-se por: *4T (decodifica por meio de uma tabela).

7

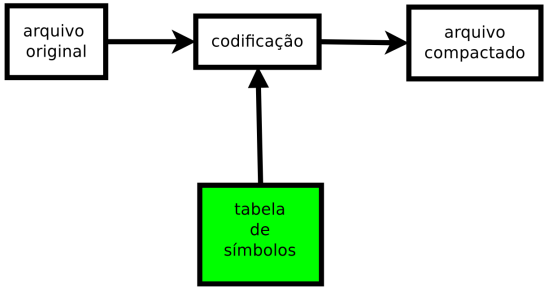
Desvantagens

- Custo de processamento na compressão e na descompressão;
- Custo para armazenar a tabela de símbolos ou dicionário;
- Ganhos expressivos são obtidos apenas com métodos de compressão que não permitem reconstruir os dados exatamente da maneira como eram antes da compressão.

8

Compressão de dados

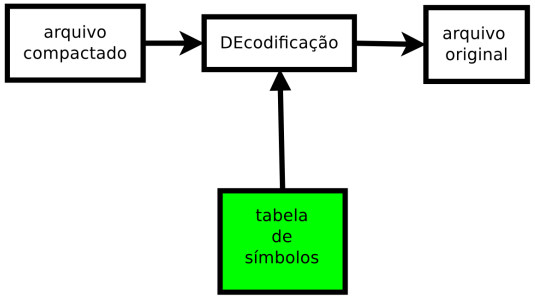
compressão: arquivo original → codificação → arquivo compactado



9

Compressão de dados

Descompressão: arquivo compactado → decodificação → arquivo original



10

Frequência de Caracteres

11

Frequência de Caracteres

- Conhecida como *Run-length encoding* (RLE).
- Usado em arquivos textos;
- Determinar a quantidade de símbolos idênticos consecutivos existentes no texto;
- Cada uma das sequências máximas de símbolos idênticos é substituída por um número indicando da frequência do símbolo em questão, seguido do símbolo

12

Frequência de Caracteres

Exemplo:

- Cadeira Original: GGCTAACAAACAAGAAACATAAACAGAATAGGCGCCCT
- Cadeira Compactada: 2G1C1T2A1C3A1C2A1G3A1C1A1T3A1C1A1G2A1T1A2G1C1G3C1
- É eficiente???
- $\frac{49}{38} \simeq 1{,}256$

13

Frequência de Caracteres

Assumir que ausência de frequência significa ocorrência = 1

Exemplo:

- Cadeira Original: GGCTAACAAACAAGAAACATAAACAGAATAGGCGCCCT
- Cadeira Compactada: 2GCT2AC3AC2AG3ACAT3ACAG2ATA2GCG3CT
- É eficiente???
- $\frac{34}{38} \simeq 0{,}895$

14

Frequência de Caracteres

- Textos que contêm números como símbolos podem causar problema, uma vez que o número pode ser confundido com frequência
- Solução: usar um caractere de escape, como @

Exemplo:

- Cadeia Original: AAA33333BA6666888DDDDDDD99999999999AABBB
- Cadeia Compactada: 3A5@3BA4@63@87D11@92A3B

15

Algoritmo de Huffman

16

Algoritmo de Huffman

- É um método de compactação que usa as probabilidades de ocorrência dos símbolos no conjunto de dados a ser compactado para determinar códigos de tamanho variável para cada símbolo.
- Idéia é usar caracteres (símbolos) com número variável de bits
- Caracteres mais comuns na mensagem são codificados com menos bits e caracteres menos comuns com mais

17

Algoritmo de Huffman

- Tem por objetivo a construção de uma árvore binária baseada na frequência de uso das letras do alfabeto de modo que as mais frequentemente utilizadas apareçam mais perto da raiz.
- Esta árvore binária é construída da baixo para cima (das folhas para a raiz) , começando a partir das letras menos usadas até atingir a raiz.

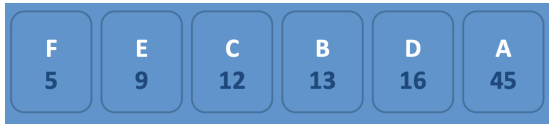
18

Algoritmo de Huffman

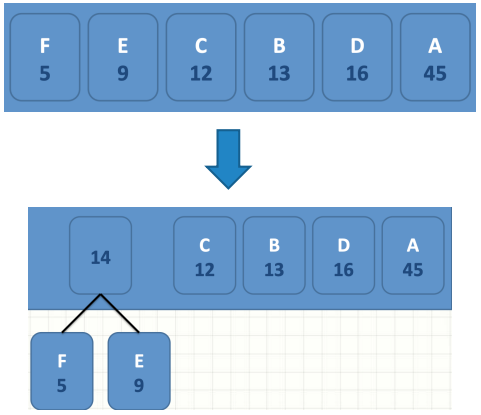
Sequência de caracteres:

FAAFEEEAAAAEEEECCAAAAAAAAACFFCCAAAACCCBAA
ABBBBAAAAAAAADDDDBBBBBBDDDDAAAAAADDDDD
DAAAAEE

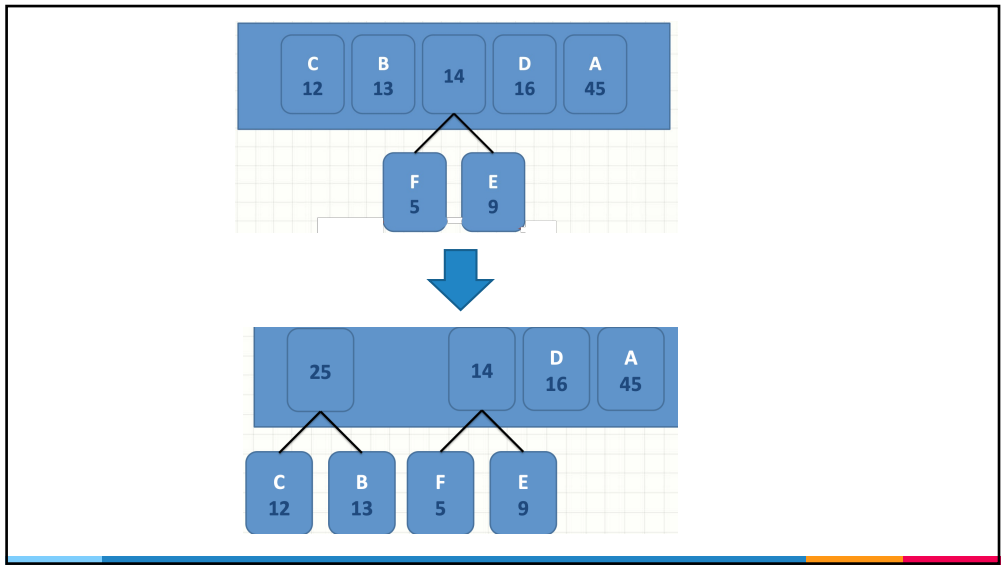
- Caracteres: FECBDA
- Frequência: 5 9 12 13 16 45 (respectivamente)



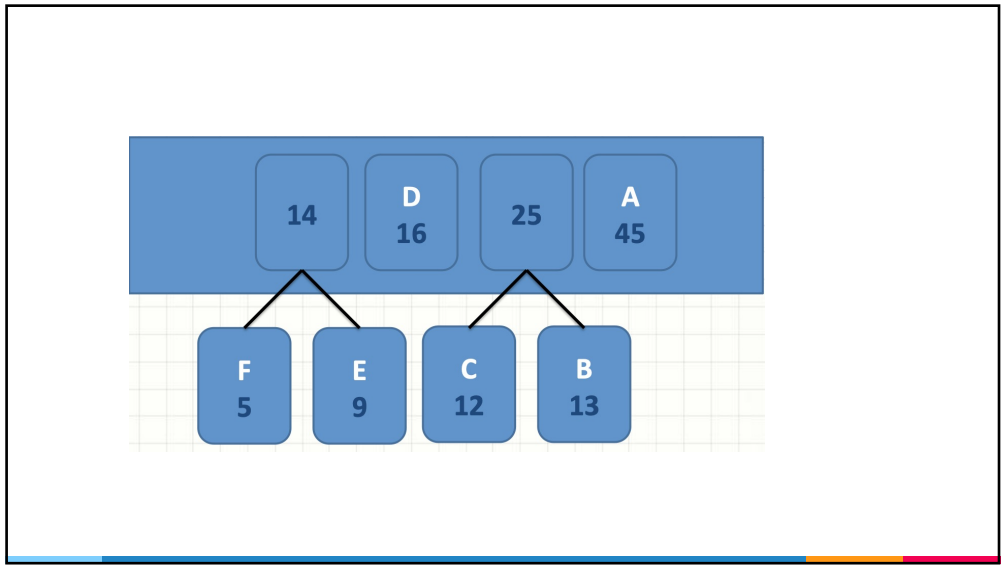
19



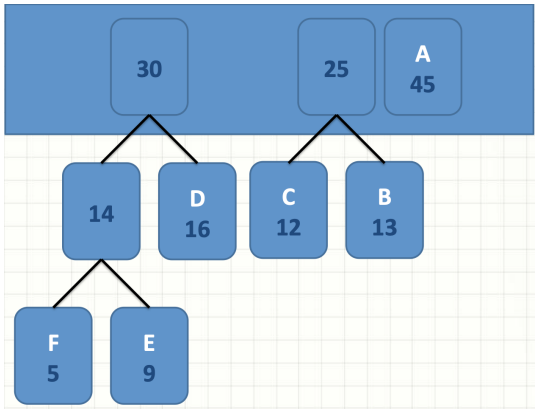
20


C
12
B
13
14
D
16
A
45
F
5
E
9
25
14
D
16
A
45
C
12
B
13
F
5
E
9

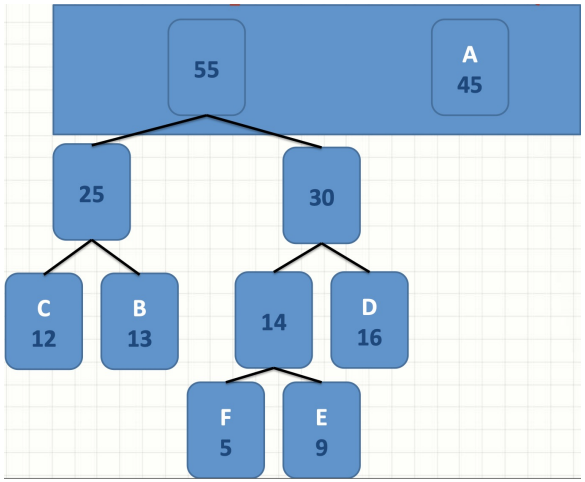
21


14
D
16
25
A
45
F
5
E
9
C
12
B
13

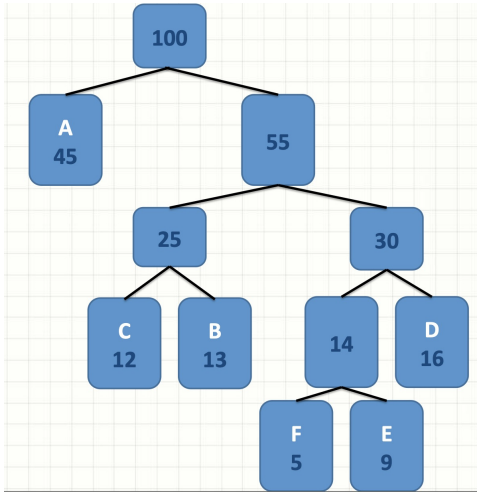
22



23



24

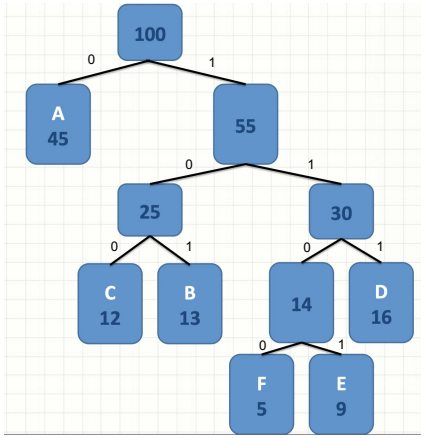


25

A tabela de codificação resultante

Caracter	Huffman
A	0
C	100
B	101
D	111
F	1100
E	1101

26



Caracter	Huffman
A	0
C	100
B	101
D	111
F	1100
E	1101

27

A tabela de codificação resultante

Caracter	Huffman	ASCII
A	0	0100 0001
C	100	0100 0011
B	101	0100 0010
D	111	0100 0100
F	1100	0100 0110
E	1101	0100 0101

28

Codificação de Huffman

Sem compactação (ASCII) :

0100011001000001010000010100011001000101010001010100010101000001010000010100000101000001
0100000101000101010001010100010101000101010000110100001101000001010000010100000101000001
0100000101000001010000010100000101000001010000010100000101000011010001100100011001000011
0100001101000001010000010100000101000001010000110100001101000011010000100100000101000001
0100000101000001010000100100001001000010010001001000001010000010100000101000001010000010
1000001010000010100000101000001010010001000100010001000100010001000100010010000100100001001
0000100100001001000010010000100100010001000100010001000100010001000100010000010100000101
0000010100000101000001010000010100010001000100010001000100010001000100010001000100010001
0000010100000101000001010000010100001010100010101000101

Com compactação

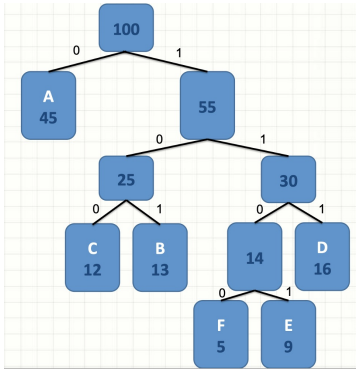
1100001100110111011101000011011101110111011101100100000000000100110011001001000000010
0100100101000101101101101000000001111111111111101101101101101101111111111111100000
0111111111111111 111000011011101

29

Decodificação

Para decodificar uma mensagem obtida, basta ir utilizando cada bit da mensagem para percorrer a arvore de Huffman desde a raiz até alguma folha, quando se obtém o símbolo decodificado. Volte então para a raiz e continue a percorrer a árvore para decodificar o próximo símbolo.

30



- 1100001100110111011101000011011
  1011101110110010000000000010011
  0011001001000000100100100101000
  1011011011010000000011111111111
  1101101101101101101111111111111
  0000001111111111111111110000110
  11101
- FAAFEE

31

LZW (*Lempel-Ziv-Welch*)

32

LZW

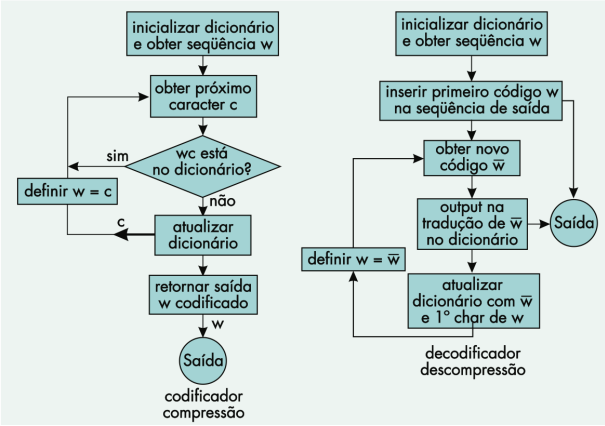
- Procura substituir sequências de símbolos por códigos.
- Para obter a compressão de dados, os códigos devem ser menores que as sequências representadas por eles.
- Na prática, esse algoritmo consegue um maior nível de compressão de dados que os outros métodos
- O LZW é usado para compactar arquivos binários em geral, a exemplo das imagens, vídeos e dos textos

33

LZW

- Huffman é um método estatística de compressão de dados.
- LZW é um método baseado em dicionários
- Diferentemente do algoritmo de Huffman, o algoritmo LZW transforma strings de tamanho variado em códigos de tamanho fixo de W bits (usualmente de 8 a 12).

34



35

Codificação

1. No início o dicionário contém todas as raízes possíveis e P é vazio;
2. C <= próximo caracter da sequência de entrada;
3. A string P+C existe no dicionário ?
   a. se *sim*,
      i. P <= P+C;
   b. se *não*,
      i. coloque a palavra código correspondente a P na sequência codificada;
      ii. adicione a string P+C ao dicionário;
      iii. P <= C;
4. Existem mais caracteres na sequência de entrada ?
   a. se *sim*,
      i. volte ao passo 2;
   b. se *não*,
      ii. coloque a palavra código correspondente a P na sequência codificada;
      iii. FIM.

36

1. No início o dicionário contém todas as raízes possíveis e P é vazio;
2. C <= próximo caracter da sequência de entrada;
3. A string P+C existe no dicionário ?
   a. se *sim*,
      i. P <= P+C;
   b. se *não*,
      i. coloque a palavra código correspondente a P na sequência codificada;
      ii. adicione a string P+C ao dicionário;
      iii. P <= C;
4. Existem mais caracteres na sequência de entrada ?
   a. se *sim*,
      i. volte ao passo 2;
   b. se *não*,
      ii. coloque a palavra código correspondente a P na sequência codificada;
      iii. FIM.

Posição	1	2	3	4	5	6	7	8	9
Caracter	a	b	b	a	b	a	b	a	c

37

1. No início o dicionário contém todas as raízes possíveis;
2. cW <= primeira palavra código na sequência codificada (sempre é uma raiz);
3. Coloque a string(cW) na sequência de saída;
4. pW <= cW;
5. cW <= próxima palavra código da sequência codificada;
6. A string(cW) existe no dicionário ?
   a. se *sim*,
      i. coloque a string(cW) na sequência de saída;
      ii. P <= string(pW);
      iii. C <= primeiro caracter da string(cW);
      iv. adicione a string P+C ao dicionário;
   b. se *não*,
      i. P <= string(pW);
      ii. C <= primeiro caracter da string(pW);
      iii. coloque a string P+C na sequência de saída e adicione-a ao dicionário;
7. Existem mais palavras código na sequência codificada ?
   a. se *sim*,
      i. volte ao passo 4;
   b. se *não*,
      i. FIM.

DECODIFICAÇÃO

38

Organização de Arquivos

39

Arquivos

- Ao construir uma estrutura de arquivos, estamos impondo uma organização aos dados
- Armazenamento de grande volume de dados e/ou aumento da complexidade dos acessos
  - Problemas de eficiência no armazenamento dos arquivos e dos acessos aos registros
  - Solução: Técnicas sofisticadas de armazenamento e recuperação de dados

40

Organização de Arquivos

- Quando um arquivo é desenhado ou construído, as qualidades esperadas dele dizem respeito à:
- Rapidez na recuperação da informação.
- Facilidade para atualizar informações.
- Pouco consumo de área para o armazenamento da informação.
- O armazenamento é uma tarefa extremamente simples.
- Já a recuperação de arquivos, nem TANTO.

41

Organização de Arquivos

Na recuperação de arquivos devemos observar:

- O uso de índices adequados, a fim de possibilitar a execução de consultas com maior rapidez.

Considerar todo e qualquer tempo:

- Tempo de demora em obter a informação.
- Tempo de inserção
- Tempo de atualização.
- Tempo de realocação e memória gasta.

Para observar todas essas variáveis, a escolha de um método de acesso adequado se torna a peça-chave nesse contexto.

42

Terminologia

Arquivo
- Coleção de registros lógicos, cada um representando um objeto ou entidade

Registro Lógico (Registro)
- Seqüência de itens, sendo cada item campo ou atributo
- Cada campo representa uma característica ou propriedade, possui nome, tipo e comprimento
- Comprimento pode ser constante ou variável

Registro Físico
- Armazenamento do arquivo em bloco de registros lógicos chamado registro físico (leitura/gravação)
- Tamanho do bloco coincide com uma unidade de armazenamento do meio físico (Ex.: setor, trilha)
- Cada bloco armazena um número inteiro de registros

43

Terminologia

Chave:
- Sequencia de um ou mais campos de um registro

Chave Primária;
- É uma chave que apresenta um valor diferente para cada registro do arquivo

Chave Secundária:
- Difere de uma chave primária pela possibilidade de ter seu valor repetido em diferentes registros

Chave de Acesso
- É uma chave utilizada para identificar o(s) registro(s) desejado(s) em uma operação de acesso a um arquivo

44

Métodos de Acesso

A escolha da metodologia de acesso adequado determinará o grau de dificuldade a ser encontrado em um sistema de informações.

São eles:

- Acesso Serial
- Acesso Sequencial
- Acesso Sequencial Indexado
- Acesso Direto

45

Acesso Serial

- Tem como característica o armazenamento dos registros de acordo com a ordem arbitrária determinada pelo programa gerador, inexistindo, assim, uma seqüência lógica na gravação.
- Outra característica, é que todos os registros são do mesmo tamanho físico, portanto, os campos e valores dentro do registro têm a mesma seqüência e posição relativa.
- Vantagem: simplicidade
- Desvantagens: tempo de busca, atualização exige a busca, indexação gera outro arquivo.

46

Acesso Sequencial

Os registros estão dispostos ordenadamente, obedecendo à sequencia determinada por uma chave primária, chamada de chave de ordenação.

EMPREGADO

Matricula	Nome	Idade	Salário
1000	Ademar	32	5000
1010	Roberto	25	7500
1020	Gerson	43	6000
1030	Yeda	23	9000
1040	Bernardo	21	4500
1050	Ângela	29	5000

47

Acesso Sequencial

- Lê o arquivo registro a registro, em busca de um registro contendo um certo valor de chave
- Uma busca por um registro em um arquivo com 2.000 registros
- Requer, em média, 1.000 leituras
- 1 leitura se for o primeiro registro,
- 2.000 se for o último
- 1.000, em média (supondo igual probabilidade de busca por qualquer registro)
- No pior caso, o trabalho necessário para buscar um registro em um arquivo de tamanho n utilizando busca sequencial é O(n)

48

Acesso Sequencial Indexado

- Um dos mais utilizados para o arquivamento de dados, pode se tratar de uma metodologia que oferece como vantagem o fato de ser mais rápida na seleção de informações sem a necessidade de ler o arquivo de forma sequencial.
- O acesso direto ao registro é feito por meio do índice criado durante a geração do arquivo e/ou durante a manutenção deste.

49

Acesso Direto

- Acesso rápido aos registros especificados por argumentos de pesquisa, sem percorrer uma estrutura auxiliar (índice).
- Em um arquivo direto, ao invés de um índice é utilizada uma função (hashing) que calcula o endereço do registro a partir do valor da chave do registro.

50

Conteúdos da prova

51

Conteúdo prova

- Grafos:
  - Principais conceitos;
  - Matriz e lista de adjacência;
  - Buscas em grafos (BFS e DFS);
  - Caminhos mínimos (Dijkstra e Bellman-Ford)
  - Árvore geradora mínima (Prim e Kruskal)
- Compressão de dados e organização de arquivos.

52